# LABORATORY OBSERVATION/RECORD

**Course:** Full Stack Web Development
**Course Code:** 23CM4121
**Experiment No.:** 1
**Student Name:** Ch.Gouri Sai Sree
**Roll No.:**A24126552076
**Date:** 25-08-2026

## 1.TITLE

**Design Static Webpage using HTML Components**

## 2. AIM

To design and develop a static webpage using various HTML components. To understand the structure and usage of HTML elements for creating a well-organized webpage.

## 3. OBJECTIVE

The objectives of this experiment are:

- To understand the basic structure and components of HTML.
- To design a static webpage using various HTML elements.
- To use headings, paragraphs, lists, tables, links, images, and forms.
- To organize webpage content using appropriate HTML components.
- To create a simple and well-structured static webpage.

## 4. THEORY

**HTML (HyperText Markup Language)** is the standard markup language used to create and structure webpages.
**HTML components/elements** are used to organize and display different types of content on a webpage.
In this application, HTML is used to create and structure the static webpage using different components.

- **Headings** are used to display titles and headings.
- **Paragraphs** are used to display textual content.
- **Lists** are used to organize information.
- **Images** are used to display visual content.
- **Links** are used to navigate to other webpages.
- **Tables** are used to display data in rows and columns.
- **Forms** are used to collect user information.

The basic flow of the webpage is:
**HTML Components → Webpage Structure → Browser → Displayed Static Webpage**

## 5. ALGORITHM / PROCEDURE

1. Create a new HTML file.
2. Add the basic HTML document structure using <!DOCTYPE html>, <html>, <head>, and <body>.
3. Add a <header> section with the webpage title and navigation links.
4. Create navigation links using the <nav> and <a> elements.
5. Create a hero section using <section>, headings, paragraphs, and links. 6. Create the main content area using <main>.
7. Add articles using the <article> element with headings and paragraphs. 8. Create an interactive form using <form>, <fieldset>, and <legend>. 9. Add form controls such as text fields, email input, dropdown, textarea, submit, and reset buttons.
10. Create a sidebar using the <aside> element.
11. Add unordered and ordered lists to display information.
12. Create a footer section using the <footer> element.
13. Use tables for structural layout and alignment of webpage content.
14. Save the HTML file and open it in a web browser.
15. Verify the webpage structure, navigation links, form elements, and displayed output.

# 6. PROGRAM

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Pure Semantic HTML Static Webpage</title>
</head>
<body>
```

```
    <header>
        <table width="100%" border="0" cellspacing="0" cellpadding="10">
            <tr>
                <td>
                    <h1>StaticWeb</h1>
                </td>
                <td align="right">
                    <nav>
                        <a href="#home">Home</a>  | 
                        <a href="#articles">Articles</a>  | 
                        <a href="#about">About</a>  | 
                        <a href="#contact">Contact Us</a>
                    </nav>
                </td>
            </tr>
        </table>
    </header>

    <hr>

    <section id="home">
        <center>
            <h2>Semantic Structures Without CSS</h2>
            <p>This layout uses raw HTML components to build an organized document
hierarchy that browsers can cleanly parse out-of-the-box.</p>
            <p><a href="#articles"><b>Read Our Articles Below &darr;</b></a></p>
        </center>
    </section>

    <hr>

    <table width="100%" border="0" cellspacing="0" cellpadding="20">
        <tr valign="top">

            <td width="70%">
                <main id="articles">

                    <article>
                        <h2>1. Core Structural Tags</h2>
                        <p>Published: July 7, 2026</p>
                        <p>When you omit cascading style sheets entirely, the
semantic markup handles all of the structural heavy lifting. Elements like
<code>&lt;main&gt;</code>, <code>&lt;article&gt;</code>, and
<code>&lt;section&gt;</code> tell screen readers and web scrapers exactly how
```

```
information flows.</p>
                        <p>Using raw browser defaults ensures excellent loading
performance, perfect accessibility compliance, and absolute structural clarity.</p>
                    </article>

                    <br><hr><br>

                    <article>
                        <h2>2. Native Interaction Elements</h2>
                        <p>Published: July 5, 2026</p>
                        <p>Browsers come equipped with excellent interactive elements
by default. We can capture text inputs, provide multiple-choice configurations, and
submit structured information to backend parsers using pure markup.</p>

                        <h3>Interactive Demonstration Forms</h3>
                        <form id="contact" action="#" method="post">
                            <fieldset>
                                <legend><b>User Inquiry Form</b></legend>
                                <p>
                                    <label for="name">Full Name: </label>
                                    <input type="text" id="name" name="username"
required placeholder="John Doe">
                                </p>
                                <p>
                                    <label for="email">Email Address: </label>
                                    <input type="email" id="email" name="useremail"
required>
                                </p>
                                <p>
                                    <label for="topic">Inquiry Topic: </label>
                                    <select id="topic" name="usertopic">
                                        <option value="general">General
Feedback</option>
                                        <option value="technical">Technical
Architecture</option>
                                        <option value="academics">Academic
Curriculum</option>
                                    </select>
                                </p>
                                <p>
                                    <label for="message">Message Details:</label><br>
                                    <textarea id="message" name="usermessage"
rows="5" cols="40" placeholder="Type your notes here..."></textarea>
                                </p>
                                <p>
```

```
                                    <input type="submit" value="Submit Form Data">
                                    <input type="reset" value="Clear Entries">
                                </p>
                            </fieldset>
                        </form>
                    </article>

                </main>
            </td>

            <td width="30%" bgcolor="#f4f4f4">
                <aside id="about">
                    <h3>Document References</h3>
                    <p>Static pages map data out directly without real-time database queries or processor overhead.</p>

                    <h4>Core Benefits:</h4>
                    <ul>
                        <li>Instant browser painting</li>
                        <li>Low memory footprints</li>
                        <li>High data compatibility</li>
                    </ul>

                    <h4>Required Components:</h4>
                    <ol>
                        <li>Document Type Definition</li>
                        <li>Semantic Wrapper Blocks</li>
                        <li>Data Input Control Interfaces</li>
                    </ol>
                </aside>
            </td>

        </tr>
    </table>

    <hr>

    <footer>
        <center>
            <p>&copy; 2026 Pure HTML Architecture. Assembled completely without style configurations.</p>
        </center>
    </footer>
```

```
</body>
</html>
```

# 7. CODE EXPLANATION

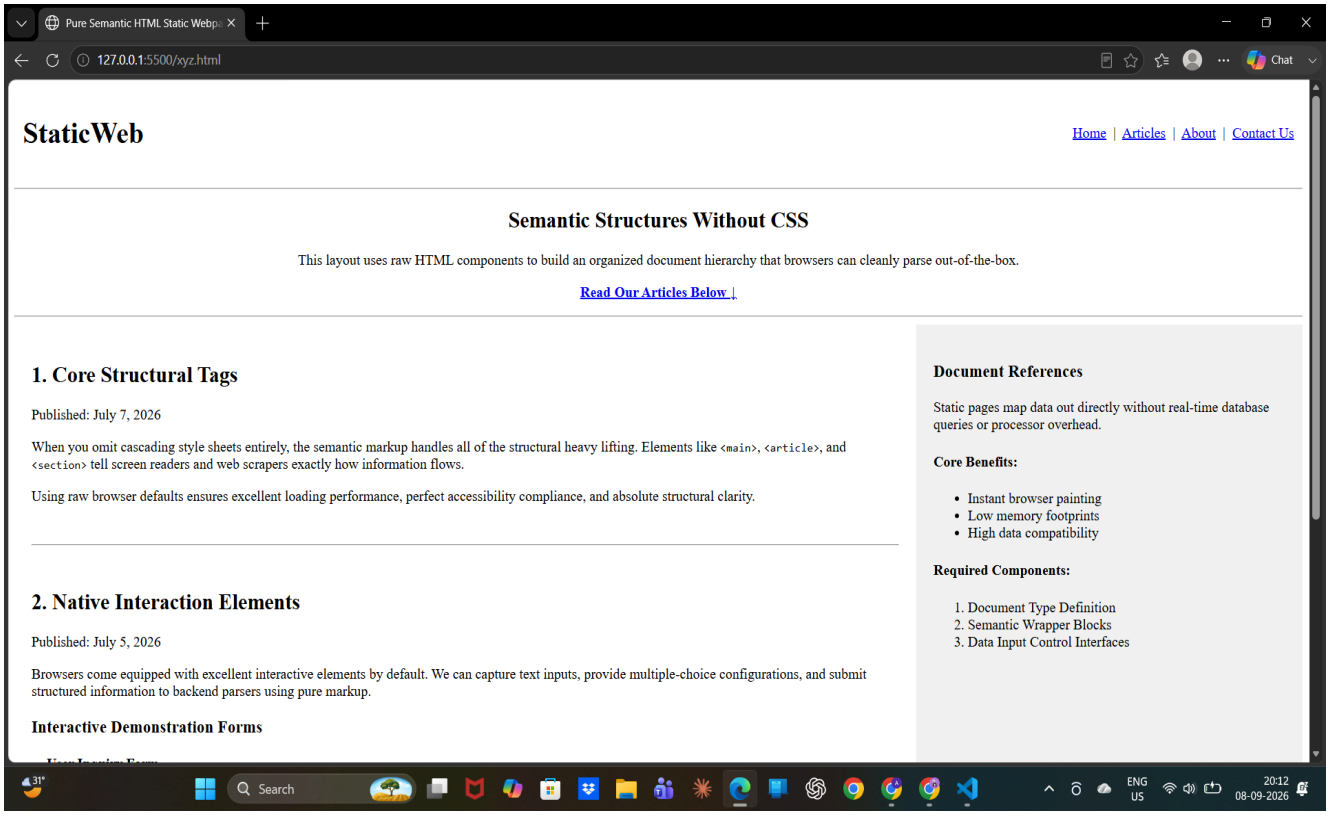
| Code | Explanation |
|---|---|
| <!DOCTYPE html> | Defines the document as an HTML5 webpage. |
| <html> | Represents the root element of the HTML document. |
| <head> | Contains metadata and the title of the webpage. |
| <body> | Contains all the visible content of the webpage. |
| <header> | Defines the header section of the webpage. |
| <nav> | Contains navigation links such as Home, Articles, About, and Contact Us. |
| <section> | Defines a separate section of the webpage. |
| <main><br><article><br><form> | Represents the main content of the webpage.<br>Defines independent article/content sections.<br>Creates a form for collecting user information. |
| <input> | Creates input fields such as text, email, submit, and reset controls. |
| <select> | Creates a dropdown list for selecting an inquiry topic. |
| <textarea> | Provides a multi-line input area for messages. |
| <aside> | Defines the sidebar containing additional information. |
| <ul> / <ol> | Creates unordered and ordered lists. |
| <footer> | Defines the footer section of the webpage. |
| <table> | Organizes and aligns webpage content into rows and columns. |

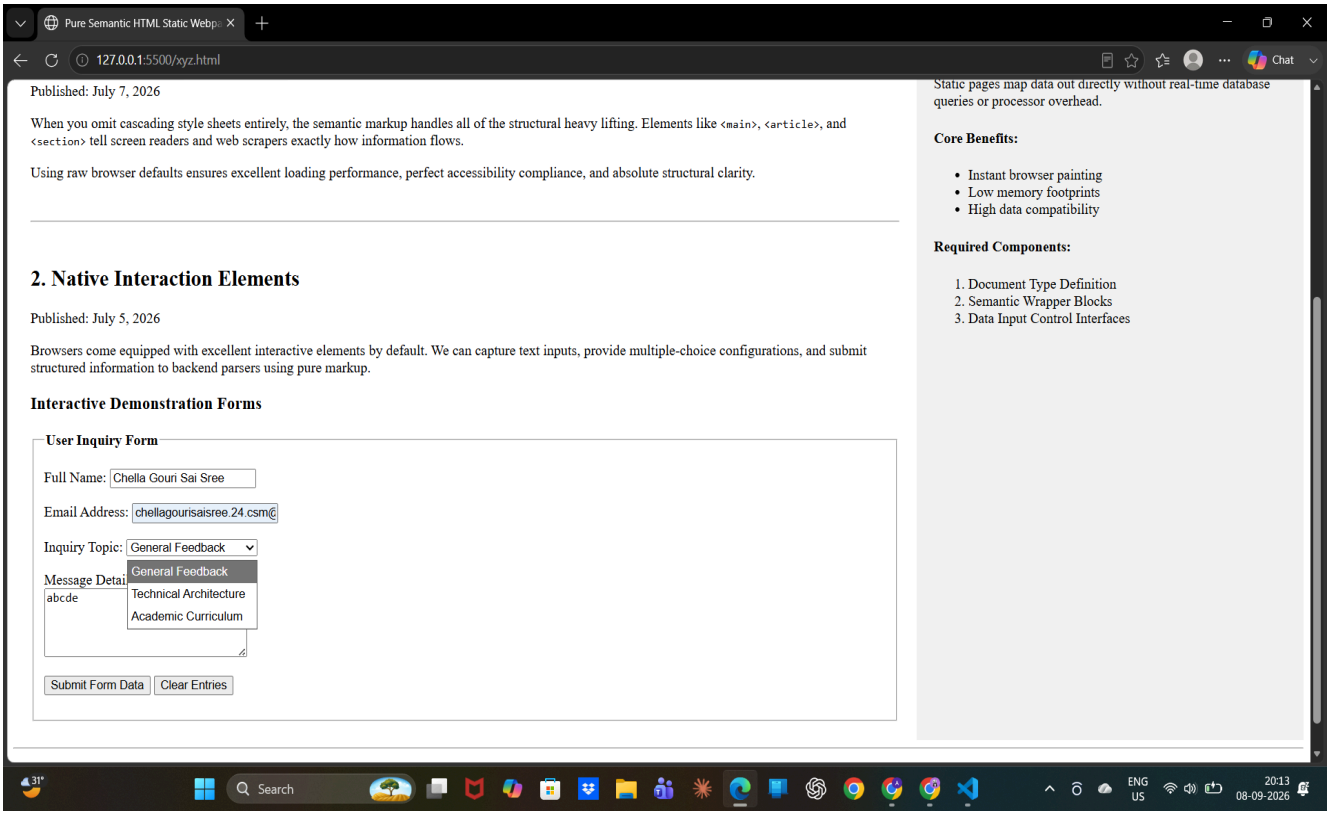
| | |
|---|---|
| <a href=""> | Creates hyperlinks for navigation within the webpage. |

# 8. TEST CASES AND EXECUTION DATA

| Test Case | Test Scenario | Input / Action | Expected Result | Actual Result | Status |
|---|---|---|---|---|---|
| TC-01 | Open webpage | Open HTML file in browser | Webpage loads correctly | Webpage loaded correctly | Pass |
| TC-02 | Check navigation links | Click Home, Articles, About, Contact Us | Navigates to the respective sections | Navigated correctly | Pass |
| TC-03 | Enter valid form data | Name: Rahul, Email: rahul@gmail .com | Data should be accepted in the form | Data entered successfully | Pass |
| TC-04 | Select inquiry topic | Select "Technical Architecture" | Selected option should be displayed | Option selected correctly | Pass |
| TC-05 | Submit form | Click "Submit Form Data" | Form should be submitted | Form submitted successfully | Pass |
| TC-06 | Reset form | Enter data andclick "Clear Entries" | All form fieldsshould becleared | Fields cleared successfully | Pass |
| TC-07 | Check webpage sections | Verify Header, Main, Aside andFooter | All sections shouldbe displayed properly | All sections displayed correctly | Pass |

# 9. OUTPUT


Pure Semantic HTML Static Webp
127.0.0.1:5500/xyz.html
Chat
StaticWeb
Home | Articles | About | Contact Us
Semantic Structures Without CSS
This layout uses raw HTML components to build an organized document hierarchy that browsers can cleanly parse out-of-the-box.
Read Our Articles Below ↓
1. Core Structural Tags
Published: July 7, 2026
When you omit cascading style sheets entirely, the semantic markup handles all of the structural heavy lifting. Elements like <main>, <article>, and <section> tell screen readers and web scrapers exactly how information flows.
Using raw browser defaults ensures excellent loading performance, perfect accessibility compliance, and absolute structural clarity.
2. Native Interaction Elements
Published: July 5, 2026
Browsers come equipped with excellent interactive elements by default. We can capture text inputs, provide multiple-choice configurations, and submit structured information to backend parsers using pure markup.
Interactive Demonstration Forms
Document References
Static pages map data out directly without real-time database queries or processor overhead.
Core Benefits:
Instant browser painting
Low memory footprints
High data compatibility
Required Components:
1. Document Type Definition
2. Semantic Wrapper Blocks
3. Data Input Control Interfaces
Search
ENG US
20:12
08-09-2026


Pure Semantic HTML Static Webp
127.0.0.1:5500/xyz.html
Chat
Published: July 7, 2026
When you omit cascading style sheets entirely, the semantic markup handles all of the structural heavy lifting. Elements like <main>, <article>, and <section> tell screen readers and web scrapers exactly how information flows.
Using raw browser defaults ensures excellent loading performance, perfect accessibility compliance, and absolute structural clarity.
2. Native Interaction Elements
Published: July 5, 2026
Browsers come equipped with excellent interactive elements by default. We can capture text inputs, provide multiple-choice configurations, and submit structured information to backend parsers using pure markup.
Interactive Demonstration Forms
User Inquiry Form
Full Name: Chella Gouri Sai Sree
Email Address: chellagourisaisree.24.csm@
Inquiry Topic: General Feedback
General Feedback
Technical Architecture
Academic Curriculum
Message Detai
abcde
Submit Form Data
Clear Entries
Document References
Static pages map data out directly without real-time database queries or processor overhead.
Core Benefits:
Instant browser painting
Low memory footprints
High data compatibility
Required Components:
1. Document Type Definition
2. Semantic Wrapper Blocks
3. Data Input Control Interfaces
Search
ENG US
20:13
08-09-2026

# 10. RESULT

Thus, a **static webpage** was successfully designed using **HTML components**. Various semantic elements, navigation links, articles, forms, lists, tables, and footer sections were implemented successfully. The webpage was tested in a browser and verified using multiple test cases.